

Low-Level Design (LLD) - Server Latency Monitor

(Python Core & Pandas DataFrames)

Difficulty Level: Medium | Total Marks: 20

Design Format: 1 Class | 7 Methods | 7 Test Cases

****(Marks allocation details in "Test Cases & Marks Allocation" section)****

Summary of Design Requirements

Implement a class 'ServerLatencyMonitor' to track and analyze server latency readings using Pandas DataFrames. The class focuses on data cleaning, identify peak health readings, and calculating medical trends.

The class should:

- Load server latency logs from a CSV file.
- Handle missing latency values by cleaning the dataset.
- Provide statistical insights such as averages and extreme values.
- Monitor risk levels by identifying and counting high-latency endpoints.

Concepts Tested

- DataFrame state management and schemas
- Data cleaning (NaN handling)
- Aggregation and Grouping (max, groupby, mean)
- Boolean indexing and filtering
- Unique value extraction and counting

Problem Statement

Design a health monitoring tool 'ServerLatencyMonitor' to analyze server latency data. The system starts by loading 'latency.csv' containing server metrics. The monitor must be capable of cleaning incomplete records, identifying peak readings, and generating reports on endpoint averages and risk categories.

Note: The foundational methods for initialization and data loading (Method 1 & 2) are pre-provided.

Operations (Methods)

1. Initialize Monitor [0 Marks]

def __init__(self):

- Logic: Initialize 'self.df = None'.

2. Read Heart Data [0 Marks]

def read_data(self, file_path: str):

- Logic: Load the CSV file into the Pandas DataFrame 'self.df'.

3. Clean Records [4 Marks]

def clean_records(self) -> int:

- Logic: Drop rows where 'Latency' is NaN. Update the internal DataFrame.

- Return: The number of rows removed.

4. Find Highest Latency [4 Marks]

def find_highest_latency(self) -> int:

- Logic: Find the maximum latency value across all cleaned records.
- Return: The peak integer latency.

5. Average Latency per Endpoint [4 Marks]

def endpoint_averages(self) -> dict:

- Logic: Calculate mean 'Latency' for each 'EndpointID' (rounded to 2 decimal places).
- Return: Dictionary with 'EndpointID' as keys and average latencies as values.

6. High Latency Endpoints [4 Marks]

def high_latency_endpoints(self, threshold: int) -> list:

- Logic: Identify unique 'EndpointID' values who have at least one reading above 'threshold'.
- Return: Sorted list of unique high-latency Endpoint IDs.

7. Count High Latency Endpoints [4 Marks]

def count_high_latency_endpoints(self, threshold: int) -> int:

- Logic: Count how many unique endpoints meet the high-latency criteria (> 'threshold').
- Return: Integer count of high-latency endpoints.

Test Cases & Marks Allocation

Test Case ID	Description	Method	Marks
TC1	Verify initial state (Sample)	__init__	0
TC2	Verify data loading (Sample)	read_data()	0
TC3	Verify removal of missing values	clean_records()	4
TC4	Identify highest recorded latency	find_highest_latency()	4
TC5	Calculate averages per endpoint	endpoint_averages()	4
TC6	Identify endpoints exceeding 200 ms	high_latency_endpoints(200)	4
TC7	Count unique high-latency endpoints	count_high_latency_endpoints(200)	4

Visible Test Case Descriptions

- TC1: self.df is None upon class creation.
- TC2: read_data() loads the sample data into self.df (12 rows initially).
- TC3: clean_records() returns 2 (Removing records for E02 and E06).
- TC4: find_highest_latency() returns 500.
- TC5: endpoint_averages()['E01'] returns approximately 120.33.
- TC6: high_latency_endpoints(200) returns ['E03'].
- TC7: count_high_latency_endpoints(200) returns 1.